Kubernetes Deployment Project

Deploying a Scalable Web Application on Kubernetes

1. Objective

Deploy a containerised web application on a Kubernetes cluster with multiple replicas, expose it through a Service, scale it up and down, and perform a rolling update with zero downtime. The project covers the core Kubernetes objects: Pod, Deployment, Service, ConfigMap and Ingress.

2. Architecture

Users reach the application through a Service (NodePort or LoadBalancer). The Service forwards traffic to Pods managed by a Deployment. If a Pod crashes, the Deployment automatically creates a new one.

Object	Purpose
Deployment	Runs and manages 3 replicas of the app
Pod	Smallest unit; runs the container
Service	Stable network endpoint that load balances Pods
ConfigMap	Stores configuration separate from the image
Ingress (optional)	Host and path based routing from outside
HPA (optional)	Autoscaling based on CPU usage

3. Prerequisites

• Docker installed, and a cluster: **minikube** or **kind** (local), or Amazon EKS.

• **kubectl** command-line tool.

• A Docker image of your app (this guide uses nginx as the sample).

4. Step-by-Step Implementation

Step 1: Start the cluster

```
minikube start --driver=docker
kubectl get nodes
```

Step 2: Create a namespace

```
kubectl create namespace demo
```

Step 3: Create a ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: demo
data:
  APP_MODE: "production"
```

```
  GREETING: "Hello from Kubernetes"
```

Step 4: Create the Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: demo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web
        image: nginx:1.25
        ports:
        - containerPort: 80
        envFrom:
        - configMapRef:
            name: app-config
        resources:
          requests: { cpu: "100m", memory: "64Mi" }
          limits:   { cpu: "250m", memory: "128Mi" }
        readinessProbe:
          httpGet: { path: /, port: 80 }
          initialDelaySeconds: 5
        livenessProbe:
          httpGet: { path: /, port: 80 }
          initialDelaySeconds: 10
```

Step 5: Create the Service

```
apiVersion: v1
kind: Service
metadata:
  name: web-service
  namespace: demo
spec:
  type: NodePort
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080
```

Step 6: Apply and verify

```
kubectl apply -f configmap.yaml -f deployment.yaml -f service.yaml
kubectl get all -n demo
minikube service web-service -n demo --url
```

5. Operations and Testing

Scaling

```
kubectl scale deployment web-app --replicas=5 -n demo
```

Self-healing test

```
kubectl delete pod <pod-name> -n demo
kubectl get pods -n demo -w      # a new Pod is created automatically
```

Rolling update and rollback

```
kubectl set image deployment/web-app web=nginx:1.27 -n demo
kubectl rollout status deployment/web-app -n demo
kubectl rollout undo deployment/web-app -n demo
```

Autoscaling (needs metrics-server)

```
kubectl autoscale deployment web-app --cpu-percent=60 --min=3 --max=8 -n demo
```

6. Useful Troubleshooting Commands

Command	Use
kubectl describe pod NAME	Events and reasons for failures
kubectl logs NAME	Application logs
kubectl get events -n demo	Recent cluster events
kubectl exec -it NAME -- sh	Open a shell in the container

Common errors: **ImagePullBackOff** (wrong image name or no registry access), **CrashLoopBackOff** (app keeps failing; check logs), **Pending** (not enough resources on nodes).

7. Cleanup

```
kubectl delete namespace demo
minikube stop
```

8. Conclusion

The project shows how Kubernetes runs an application with high availability, self-healing, easy scaling and safe rolling updates. Possible extensions: Ingress with TLS, Helm charts, persistent volumes, CI/CD deployment from Jenkins, and deployment on Amazon EKS.